

Chapter 9 Efficiency of Algorithms

After studying this chapter, the student should be able to:

- Distinguish between five different common algorithms: logarithmic, linear, linear logarithmic, quadratic and exponential.
- Define the efficiency of five common algorithms.
- Understand the concept of Big-O Notation: $\log n$, n , $n \log n$, n^2 and a^n
- Assign the appropriate Big-O Notation to the five common algorithms.

9.1 DEFINITION

- To define if an algorithm is efficient, we first need to measure the efficiency. The efficiency of an algorithm depends on type of construct (sequence, decision and repetition) that is used in the algorithm. The first two (sequence and decision) do not use much computer time, but the repetition does. So, we base the efficiency of an algorithm on how many times an action is repeated.
- In our definition we use the term $f(n)$, in which n defines the parameter that control the loop (or recursion) and $f(n)$ defines how many times the loop is repeated or a recursive function is called.

9.2 COMMON ALGORITHMS

- Let us discuss the efficiency of some common algorithms.
- We discuss five common algorithms that can show the efficiency

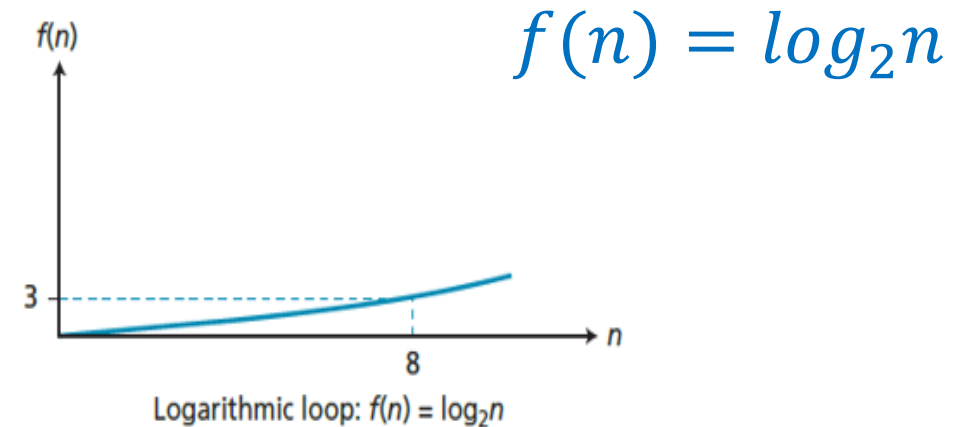
9.2.1 Logarithmic algorithm

- The first algorithm that we discuss is called logarithmic algorithm or $f(n) = \log_2 n$, in which n is the number that controls the loop and $f(n)$ defines the number of times the loop is repeated.
- The values of i in each iteration are 1, 2, 4, 8, 16, and so on. In other words, if n is 1, the loop is repeated only once, if n is 2 or 3, the loop is repeated two times. If n is 4, it is repeated three times. If n is 8, the loop is repeated 4 times. And so on. We can say that we have a **logarithmic algorithm**.

Algorithm 9.1 Logarithmic algorithm

```
int i, n; input n; i = 1;
while (i <= n)
{
    body;
    i = i * 2;
}
```

Figure 9.1 Logarithmic algorithm



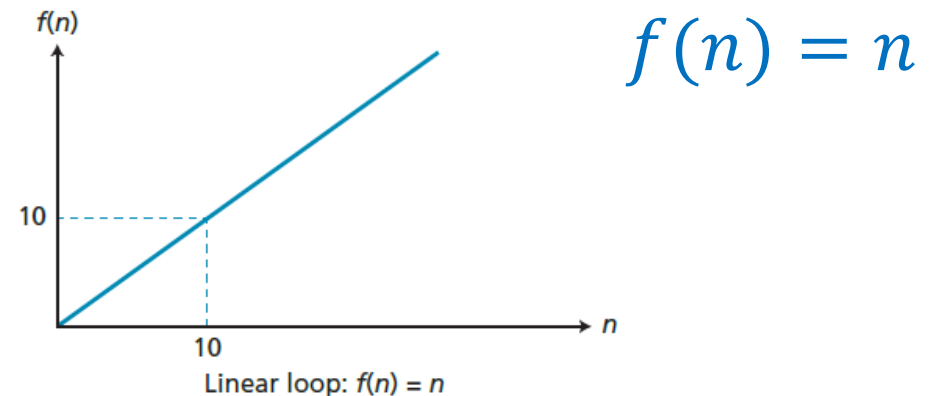
9.2.2 Linear algorithm

- The second algorithm that we discuss is called linear algorithm or $f(n) = n$, in which n is the number that controls the loop and $f(n)$ defines the number of times the loop is repeated.
- The values of i in each iteration are 1, 2, 3, 4, 5, and so on. In other words, if n is 1, the loop is repeated only once, if n is 2, the loop is repeated two times. If n is 3, it is repeated three times. And so on. We can say that we have a **linear algorithm**.

Algorithm 9.2 Linear algorithm

```
int n;  
input n;  
for (int i = 1; i <= n; i++)  
{  
    body;  
}
```

Figure 9.2 Linear algorithm



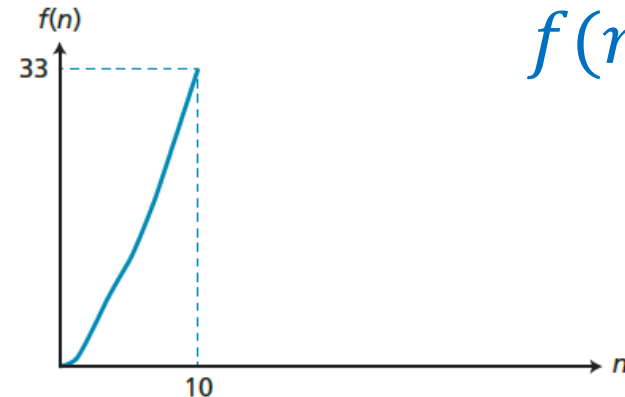
9.2.3 Linear logarithmic algorithm

- The third algorithm that we discuss is called linear logarithmic algorithm which can be shown as $f(n) = n \log_2 n$, in which n is the number that controls the loop and $f(n)$ defines the number of times the loop is repeated. In this algorithm, we normally have two loops, but the second loop depends on the first loop.
- In this case, the value of j starts from the point where i starts. In other words, the value of i goes from 1 to n , but the value of j goes from i to n .

Algorithm 9.3 Linear logarithmic algorithm

```
int n;  
input  
n;  
for (int i = 1; i <= n ; i++)  
{  
    for (int j = i; j<= n; j++)  
    {  
        body;  
    }  
}
```

Figure 9.3 Linear logarithmic algorithm



$$f(n) = n \log_2 n$$

Linear Logarithmic loop: $f(n) = n \log_2 n$

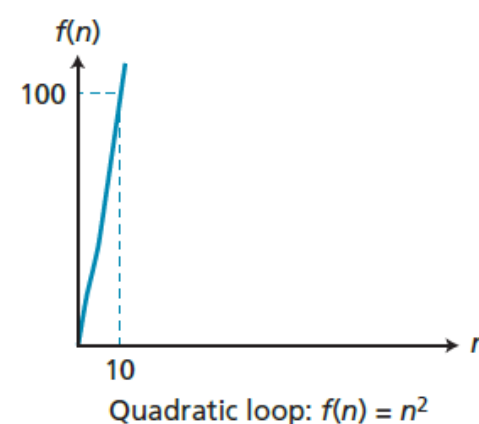
9.2.4 Quadratic algorithm

- The fourth algorithm that we discuss is called quadratic algorithm which can be shown as $f(n) = n^i$, in which n is the number that controls the loop and $f(n)$ defines the number of times the loop is repeated. In this algorithm, we normally have two loops. The second loop however can be independent of the first loop.
- Note that we have use n as the number that controls the first loop and m that controls the second loop, but n and m can be the same. In this case, we can say that we have $f(n) = n^2$.
- In other words, we have $f(n) = n.m$ which is define as n^2 .

Algorithm 9.4 Quadratic algorithm

```
int  n,
m;
input n;
input m;
for (int i = 1; i <= n ; i++)
{
    for (int j = 1; j<= m ; j++)
    {
        body;
    }
}
```

Figure 9.4 Relation between n and $f(n)$ in a quadratic algorithm



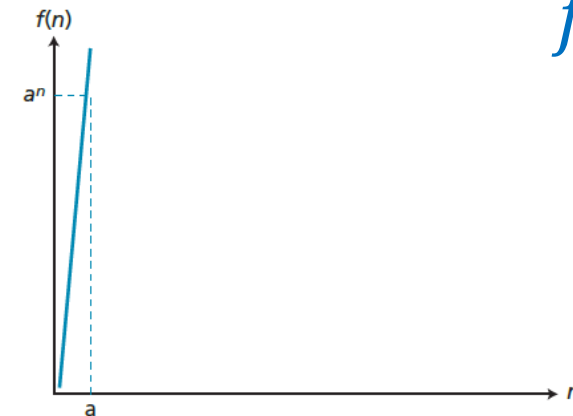
$$f(n) = n^2$$

9.2.5 Exponential

- So far, we have used the concept of repetition. Now we consider recursion. In Chapter 8, we mentioned the Fibonacci sequence. It can be defined recursively as shown below:

```
long fib (long num)
{
    if (num == 0 or num == 1)
    {
        return num;
    }
    return (fib (num - 1) + fib (num - 2) );
}
```

Figure 9.5 Exponential relation

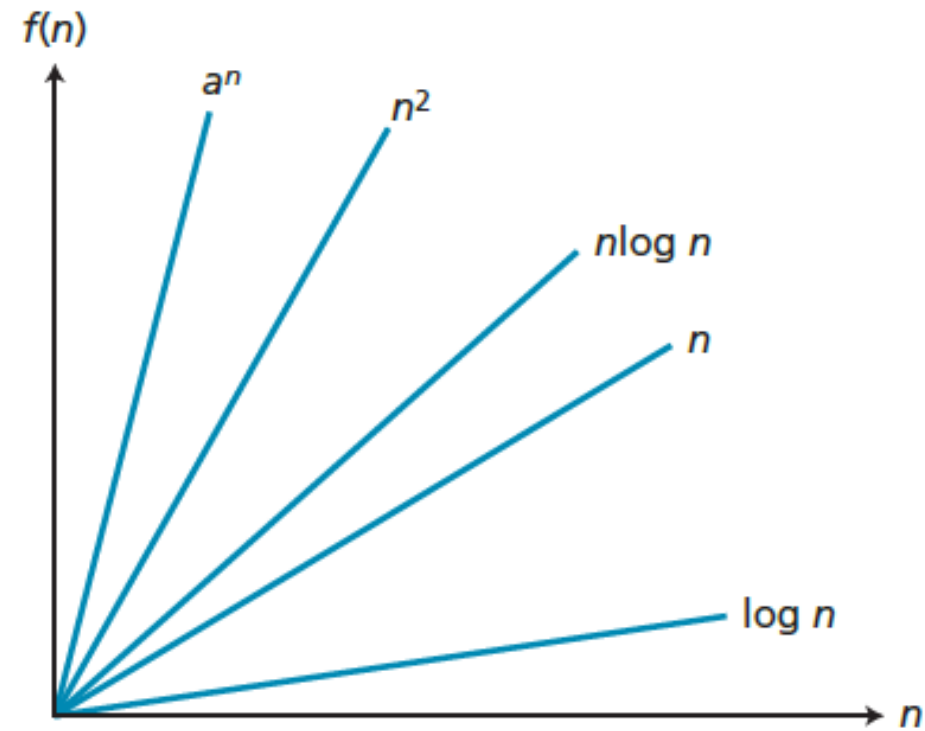


$$f(n) = a^n$$

Exponential loop: $f(n) = a^n$

- It is considered as an iteration. Although, the literature defines a normally to be 2 and the $f(n) = 2^n$, we consider the relationship as $f(n) = an$ to make it more general as shown in Figure 9.5. In the case of Fibonacci sequence, we have $f(n) = 1.6n$

Figure 9.6 *Relationship between common algorithms discussed in this chapter*



9.3 BIG-O NOTATION

- The simplification of efficiency is known as Big-O Notation as shown in Table 9.1:

Table 9.1 Standard measures

<i>Efficiency</i>	<i>Big-O</i>
Logarithmic	$O(\log n)$
Linear	$O(n)$
Linear Logarithmic	$O(n \log n)$
Quadratic	$O(n^2)$
Polynomial	$O(a^n)$

- The efficiency is increased when going down in the table. For example, the efficiency of a logarithmic algorithm is much better than a linear algorithm, and so on. This means that when we solve a problem, if we can find a solution which has a higher efficiency in the above table, we have saved computer time.

Exercises

- P.246 Problems
 - P9-5, P9-6, P9-7, P9-8, P9-9